Project Report

# A Hybrid Quantum-Classical Kernel SVM for 22-Class Crop Recommendation

Quantum Support Vector Classification with a
7-Qubit ZZ Feature Map and Fidelity Kernel

**Course:** 23CSE463 – Quantum Computing

**Submitted by**


| Student Name | Roll Number |
|---|---|
| Vishal M.D. | CH.SC.U4CSE24150 |
| Devadharshan S. | CH.SC.U4CSE24113 |

[CSE / Amrita Vishwa Vidyapeetam]


October 2026

Source code: `github.com/Devadharshan16/quantum-agri-advisory`

# Contents

**1 Title** **1**

**2 Abstract** **2**

**3 Problem Statement** **2**

**4 Objectives** **3**

**5 Proposed Algorithm** **4**

**6 Pseudocode** **7**

**7 Implementation** **7**

**8 Website / Application** **9**

**9 Results** **10**

**10 Conclusion** **16**

## 2. Abstract

Choosing a crop that suits local soil and climate is a multi-class classification problem over a handful of numeric measurements. This project investigates whether a quantum-kernel method can solve it reliably. We classify 22 crops from seven inputs (nitrogen, phosphorus, potassium, temperature, humidity, soil pH and rainfall) using a *Quantum Support Vector Classifier* (QSVC): each sample is encoded into a 7-qubit quantum state by a fixed ZZ feature map, the squared overlap of two states defines a quantum fidelity kernel, and a classical support vector machine (SVM) is trained on the resulting kernel matrix. Because the circuit has no trainable parameters, the quantum part involves no gradient-based optimisation.

Two earlier attempts motivated the final design. A 5-qubit variational classifier reached only 32.1% test accuracy, and a first kernel prototype over-fitted (100% training versus 65.0% test accuracy) while needing about 84 minutes of kernel computation on a 990-sample subset. Replacing pairwise circuit evaluation by one statevector simulation per sample, scaling the encoded features with a bandwidth parameter $\gamma$, and selecting hyper-parameters by nested cross-validation resolved both problems.

On a held-out test set of 440 samples the final model (3 repetitions, full entanglement, $\gamma = 0.1$, $C = 10$) attains **99.09%** accuracy (macro-F1 0.9909; nested-CV estimate $98.20\% \pm 0.82\%$). Tuned classical models perform equivalently: Random Forest reaches 99.55% and a linear SVM 99.09%, and McNemar's test finds no significant difference ($p = 0.617$). All quantum results are exact, noiseless simulations. The project therefore demonstrates a working and rigorously evaluated quantum-kernel pipeline; it does *not* demonstrate quantum advantage.

**Keywords:** quantum machine learning, quantum kernel, support vector machine, ZZ feature map, fidelity, kernel concentration, crop recommendation.

## 3. Problem Statement

### 3.1 The Problem

Given measurements of soil nutrients and local climate for a field, recommend the most suitable crop among 22 candidates. Crop advice is frequently generic, whereas soil and climate vary from field to field; an unsuitable crop wastes fertiliser, water and income. The task is a supervised, 22-class classification problem on low-dimensional tabular data.

A second, research-oriented question motivates the project: can a quantum machine-learning model solve such a real multi-class problem *reliably*? Trainable quantum circuits are known to be difficult to optimise (Section 9.7), and quantum kernels can fail by becoming too "sharp" to generalise. The project studies how to make a quantum-kernel classifier work, and compares it honestly with classical models.

### 3.2 Input

Each sample is a vector of seven real-valued measurements (Table 1). The dataset contains 2 200 labelled samples, 100 per crop, so the classes are perfectly balanced. The data are the publicly available *Crop Recommendation Dataset* (Kaggle, attributed to A. Ingle [13]; the file used is obtained from the public Harvestify repository).

**Table 1.** Input features. Ranges are the input-validation ranges used by the inference code, not necessarily the extrema of the raw data.

| Feature | Meaning | Unit | Validation range |
|---|---|---|---|
| `N` | Nitrogen content | kg/ha | 0–140 |
| `P` | Phosphorus content | kg/ha | 5–145 |
| `K` | Potassium content | kg/ha | 5–205 |
| `temperature` | Temperature | °C | 8–44 |
| `humidity` | Relative humidity | % | 14–100 |
| `ph` | Soil pH | – | 3.5–9.9 |
| `rainfall` | Rainfall | mm | 20–300 |

### 3.3 Expected Output

One of 22 crop labels: apple, banana, blackgram, chickpea, coconut, coffee, cotton, grapes, jute, kidneybeans, lentil, maize, mango, mothbeans, mungbean, muskmelon, orange, papaya, pigeonpeas, pomegranate, rice, watermelon. For the evaluation, the output is the predicted class of every test sample together with accuracy, macro-F1, per-class metrics and a confusion matrix.

### 3.4 Constraints and Limitations

- **Qubit budget:** the feature map uses one qubit per feature, so the circuit has 7 qubits and the state has $2^7 = 128$ amplitudes.
- **Simulation only:** all quantum computations are exact, noiseless statevector simulations on a classical computer; no quantum hardware or noise model results are reported.
- **Fixed circuit:** the feature map has no trainable parameters, so all learning happens in the classical SVM and in hyper-parameter selection.
- **Valid kernel:** the kernel matrix must be symmetric and positive semi-definite for the SVM optimisation to be convex.
- **Data:** a single static, balanced benchmark dataset of 2 200 samples (20 test samples per class); no live weather or soil feed, no missing-value handling, and the upstream licence page was not verified.
- **Evaluation:** the main results use one stratified 80/20 split (seed 42); classical baselines use a related but not identical tuning procedure.

## 4. Objectives

### 4.1 How the Problem Is Solved

We build a hybrid pipeline. A fixed 7-qubit circuit maps each sample to a quantum state; the similarity of two samples is the squared overlap of their states (a *quantum kernel*); a classical SVM then learns the decision boundaries from the matrix of pairwise similarities.

### 4.2 Algorithm Used

**Quantum Kernel Support Vector Classifier (QSVC)** with a ZZ feature map and a fidelity quantum kernel [1, 2], trained as a classical SVM [3] on a precomputed kernel matrix.

### 4.3 Approach Followed

1. Prepare the data without train–test leakage (stratified split; scalers fitted on training data only).
2. Encode every sample as a 7-qubit statevector with a ZZ feature map.
3. Compute the fidelity kernel matrix *efficiently*: one simulation per sample followed by a single matrix product.
4. Prevent over-fitting by scaling the encoded features with a bandwidth $\gamma$ and tuning feature-map repetitions, entanglement pattern and the SVM penalty $C$ by nested cross-validation.
5. Evaluate once on a held-out test set, compare against tuned classical models, and test the difference statistically.

### 4.4 How the Complexity Is Analysed

Complexity is analysed in three complementary ways: (i) *asymptotic analysis* of each stage—gate count of the feature map times the cost of applying a gate to a $2^n$-amplitude state, the cost of the kernel matrix product, and the cost of SVM training—in terms of the number of qubits $n$, repetitions $r$ and samples $N$; (ii) *exact circuit metrics* (gate counts and depth) obtained by decomposing the feature map in Qiskit; and (iii) *measured wall-clock times* from the development runs. The analysis is presented in Section 9.

## 5. Proposed Algorithm

### 5.1 Explanation of the Algorithm

**Kernel methods and the SVM.** An SVM classifies by comparing a new sample with the training samples through a kernel $\mathcal{K}(x, x')$, a similarity function that equals an inner product in some feature space [3]. For training labels $y_i$, the dual problem is

$$\max_{\alpha} \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathcal{K}(x_i, x_j) \quad \text{s.t.} \quad 0 \le \alpha_i \le C, \;\; \sum_i \alpha_i y_i = 0,$$

and a sample is classified by $f(x) = \sum_i \alpha_i y_i \mathcal{K}(x_i, x) + b$. If $\mathcal{K}$ is positive semi-definite the problem is convex and the optimum is global. For 22 classes, scikit-learn's `SVC` trains $22 \cdot 21/2 = 231$ one-vs-one binary classifiers and classifies by voting.

**Quantum feature map.** A ZZ feature map [1] encodes $x \in \mathbb{R}^n$ into the state

$$|\phi(x)\rangle = \left(U_{\Phi(x)} H^{\otimes n}\right)^r |0\rangle^{\otimes n}, \qquad U_{\Phi(x)} = \exp\Big(i \sum_{S \subseteq \{1..n\},\, |S| \le 2} \phi_S(x) \prod_{j \in S} Z_j\Big), \tag{1}$$

with $\phi_{\{i\}}(x) = x_i$ and $\phi_{\{i,j\}}(x) = (\pi - x_i)(\pi - x_j)$, as implemented by Qiskit's `ZZFeatureMap`. Here $n = 7$, $r = 3$ repetitions, and *full* entanglement, i.e. every one of the $\binom{7}{2} = 21$ qubit pairs

receives a ZZ interaction in every repetition. Single-qubit phases encode individual features; the ZZ terms encode pairwise feature interactions.

**Fidelity kernel.** The similarity of two samples is the fidelity of their encoded states,

$$\mathcal{K}(x, x') = \left| \langle \phi(x) | \phi(x') \rangle \right|^2 = \mathrm{Tr}\left[\rho(x)\rho(x')\right], \qquad \rho(x) = |\phi(x)\rangle \langle \phi(x)| . \tag{2}$$

This kernel is symmetric, satisfies $0 \leq \mathcal{K} \leq 1$ and $\mathcal{K}(x, x) = 1$, and is positive semi-definite because it is an inner product of density matrices. For a collection of encoded states stacked as the rows of a matrix $\Psi \in \mathbb{C}^{N \times 2^n}$, the whole kernel matrix is

$$K = \left|\Psi\Psi^\dagger\right|^2 \quad \text{(element-wise squared modulus)}, \tag{3}$$

so only $N$ circuits need to be simulated instead of one per pair of samples.

**Bandwidth and kernel concentration.** The features, scaled to $[0, \pi]$, are multiplied by a bandwidth $\gamma$ before encoding. If $\gamma$ is large, encoded states of different samples become nearly orthogonal and the off-diagonal kernel values collapse towards zero (*kernel concentration* [6, 7]): the kernel matrix approaches the identity, every training sample looks unique, and the SVM memorises the training set. A small $\gamma$ keeps similar samples similar. The bandwidth is therefore a central hyper-parameter.

### 5.2 Step-by-Step Process

1. **Split:** stratified 80/20 split of the 2 200 samples (1 760 training, 440 test; seed 42).
2. **Scale:** fit a `StandardScaler` and then a `MinMaxScaler` to $[0, \pi]$ on the training data only; transform the test data with these scalers and clip it to $[0, \pi]$.
3. **Bandwidth:** multiply every scaled feature by $\gamma$.
4. **Encode:** for each sample, build the 7-qubit ZZ feature-map circuit and compute its 128-amplitude statevector (Eq. 1).
5. **Kernel:** compute $K = |\Psi\Psi^\dagger|^2$ and set the diagonal to 1 (Eq. 3).
6. **Select hyper-parameters:** for every combination of repetitions $\in \{1, 2, 3\}$, entanglement $\in \{\text{full, linear, circular}\}$, $\gamma \in \{0.02, 0.05, 0.10, 0.15, 0.25, 0.50\}$ and $C \in \{0.1, 1, 10, 100, 1000\}$ (270 configurations), estimate accuracy by nested repeated stratified cross-validation on the training set only, and apply the one-standard-error rule [8].
7. **Train:** fit `SVC(kernel="precomputed")` on the selected training kernel.
8. **Evaluate:** compute the test kernel against the training states and evaluate once on the held-out test set.
9. **Predict (inference):** for a new field reading, apply the saved scalers and $\gamma$, compute its state, its kernel vector against the stored training states, and classify.

### 5.3 Flowchart

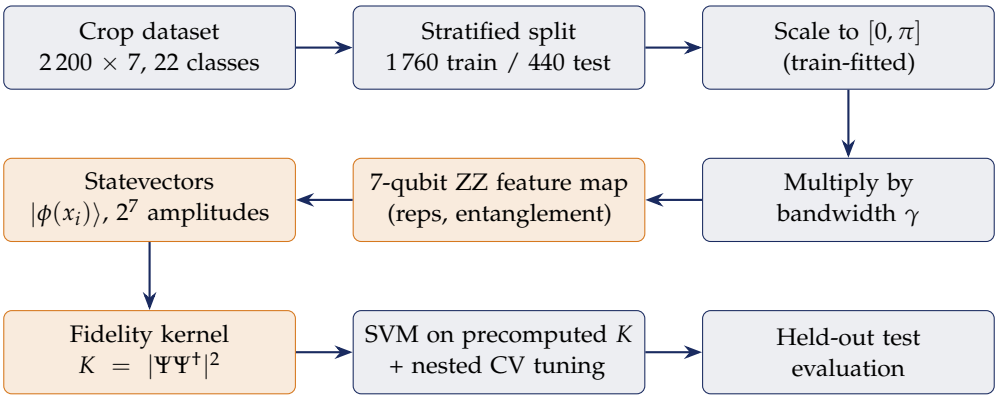



**Figure 1.** Flowchart of the proposed QSVC pipeline. Orange blocks are the quantum part (simulated), blue blocks are classical.

### 5.4 Applications of the Algorithm

Quantum-kernel SVMs were introduced as a way to classify data in a feature space defined by a quantum circuit and were first demonstrated at small scale on a superconducting-qubit processor [1]; the kernel viewpoint is developed in [2]. The same pipeline applies to any supervised classification task with a small number of numeric features, for example the crop-recommendation task studied here, other tabular decision-support problems, and benchmarking studies of quantum feature maps. In this project it serves as a decision-support classifier for choosing a crop from soil and weather measurements.

### 5.5 Why the Algorithm Is Suitable for This Problem

- **Small, low-dimensional data:** seven features map naturally to seven qubits, and $2^7 = 128$ amplitudes can be simulated exactly in milliseconds.
- **No quantum training loop:** the feature map is fixed, so the barren-plateau phenomenon associated with training deep parameterised circuits [5] does not arise in the training stage, and the SVM optimisation is convex.
- **Efficient kernel evaluation:** the fidelity kernel reduces to $N$ simulations and one matrix product (Eq. 3).
- **Established, well-understood classifier:** an SVM handles multi-class problems with a controlled capacity parameter $C$, and its generalisation can be estimated by cross-validation.
- **Caveat:** suitability here means *feasibility*. The kernel approach has its own failure mode (concentration), and the results of Section 9 show that classical models perform just as well on this dataset.

# 6. Pseudocode

```
Algorithm QSVC-Train(X, y, Grid, seed)
Input : X in R^(2200 x 7), labels y in {1..22}, Grid = {reps} x {ent} x {gamma} x {C}
Output: scalers, best configuration, trained SVM, training states Psi*

 1  (X_tr, y_tr), (X_te, y_te) <- StratifiedSplit(X, y, test = 0.2, seed)
 2  s1 <- StandardScaler.fit(X_tr);  s2 <- MinMaxScaler[0, pi].fit(s1(X_tr))
 3  X_tr <- s2(s1(X_tr));            X_te <- clip(s2(s1(X_te)), 0, pi)
 4  for each (reps, ent, gamma) in Grid:                  // 54 kernels, cached
 5      FM <- ZZFeatureMap(n = 7, reps, ent)
 6      for i = 1 .. N:
 7          |psi_i> <- Statevector( FM(gamma * x_i) )     // one circuit per sample
 8      Psi <- stack(|psi_1>, ..., |psi_N>)               // N x 2^7
 9      K   <- |Psi . Psi^dagger|^2 ;  K_ii <- 1          // fidelity kernel
10      for each C in Grid:                               // 5 values
11          score[reps, ent, gamma, C] <- NestedRepeatedCV( SVC(K, C) )
12  (reps*, ent*, gamma*, C*) <- simplest config whose score is within
                                 one standard error of the best score
13  model <- SVC(kernel = precomputed, C*).fit(K*, y_tr)
14  K_te  <- |Psi_te . Psi*^dagger|^2 ;  report accuracy(model.predict(K_te), y_te)
15  return s1, s2, (reps*, ent*, gamma*, C*), model, Psi*
```

**Listing 1.** QSVC training and model selection.

```
Algorithm QSVC-Predict(x_new)
Input : raw measurements x_new in R^7, saved s1, s2, gamma*, FM*, Psi*, model
Output: recommended crop

 1  validate that each feature of x_new lies in its allowed range
 2  x <- clip( s2(s1(x_new)), 0, pi ) * gamma*
 3  |psi_new> <- Statevector( FM*(x) )
 4  k <- | Psi* . |psi_new> |^2                          // N similarities
 5  return label( model.predict(k) )
```

**Listing 2.** QSVC inference for a new sample.

# 7. Implementation

## 7.1 Programming Language

Python 3.

## 7.2 Tools, Frameworks and Libraries

**Table 2.** Software stack (versions as declared in the project's `requirements.txt`).

| Library | Version | Use |
|---|---|---|
| Qiskit | 1.1.0 | ZZ feature map, exact statevector simulation |
| Qiskit Machine Learning | 0.7.2 | reference `FidelityQuantumKernel` (legacy prototype) |
| Qiskit Aer | 0.14.2 | simulator backend (noise-study scripts) |
| scikit-learn | 1.4.2 | preprocessing, `SVC`, cross-validation, baselines, metrics |
| NumPy | 1.26.4 | statevector and kernel-matrix algebra |
| joblib | – | parallel statevector simulation and model persistence |
| FastAPI | 0.115+ | REST `/predict` endpoint with Pydantic validation (Render) |
| Next.js | 16.4 | Production React 19 web frontend with multi-lingual UI (Vercel) |

## 7.3 Implementation Methodology

The project is organised as a sequence of scripts, each producing artefacts consumed by the next (Table 3). Development proceeded iteratively: a variational baseline, a first kernel prototype, a corrected statevector implementation (`trainv1.py`), and the final production pipeline (`trainv2.py`).

**Table 3.** Main source files.

| File | Purpose |
|---|---|
| `data_prep.py` | Downloads the CSV, encodes labels, performs the stratified split, fits the scalers on training data, saves arrays and preprocessing objects. |
| `qkernel.py` | Builds the ZZ feature map, simulates statevectors, computes fidelity kernels and kernel diagnostics. |
| `circuit_design.py` | Builds and draws the 7-qubit feature map and prints circuit metrics. |
| `trainv2.py` | Production training: cached kernels, 270-configuration search, nested repeated CV, one-standard-error rule, final fit and test evaluation. |
| `baselines.py` | Tunes and evaluates RBF-SVM, linear SVM, Random Forest and $k$-NN; runs McNemar's test. |
| `analysis.py`, `learning_curve.py` | Reports, confusion matrices, feature plots, learning and gamma curves. |
| `api.py` | FastAPI backend exposing the `/predict` endpoint for cloud consumption. |
| `web/` | Complete Next.js frontend with localized input controls and Apple-grade design. |
| `predict.py` | Command-line and batch CSV inference interface. |
| `test_qkernel.py` | Unit tests of kernel properties. |
| `train.py`, `trainv1.py` | Legacy prototype and earlier corrected version (kept for the development history). |

## 7.4 Explanation of the Important Parts of the Code

**Leakage-free preprocessing.** The `StandardScaler` and `MinMaxScaler` are fitted on the 1 760 training samples only. Test samples are transformed with these fitted scalers and clipped to $[0, \pi]$, so that no information from the test set influences preprocessing or model selection.

**Statevector simulation.** Each row is bound to the parameters of the feature-map circuit and its statevector is obtained with Qiskit's `Statevector`; chunks of samples are simulated in parallel with joblib. This yields an $N \times 128$ complex matrix $\Psi$.

**Kernel matrix.** The fidelity kernel is a single matrix product (Listing 3). For the square training kernel the matrix is symmetrised and its diagonal fixed to 1, using $\mathcal{K}_{ij} = \mathcal{K}_{ji}$ and $\mathcal{K}_{ii} = 1$. Kernel diagnostics (off-diagonal mean and standard deviation, minimum eigenvalue, effective rank) confirm that the matrix is a valid, non-degenerate kernel.

**Model selection.** Kernels are cached per $(\text{reps}, \text{entanglement}, \gamma)$ triple, giving $3 \times 3 \times 6 = 54$ kernels; the five values of $C$ reuse the same kernel. Hyper-parameters are chosen by nested

repeated stratified cross-validation [9] on the training data, and the one-standard-error rule picks the simplest configuration statistically indistinguishable from the best [8].

**Baselines and statistics.** RBF-SVM, linear SVM, Random Forest and $k$-NN are tuned by repeated stratified 5-fold cross-validation (3 repeats, seed 42) on the same split, and McNemar's test [10] compares the best classical model with the QSVC on the identical test samples.

**Reproducibility.** Seeds are fixed (default 42), the configuration, feature-map parameters, package versions and data metadata are saved with the model, kernels and statevectors are persisted, and unit tests check symmetry, unit diagonal, range $[0, 1]$ and positive semi-definiteness of the kernel.

```
def compute_states(circuit, X, n_jobs=-1, chunk_size=32):
    chunks = [X[i:i + chunk_size] for i in range(0, len(X), chunk_size)]
    parts = Parallel(n_jobs=n_jobs)(
        delayed(_encode_chunk)(circuit, c) for c in chunks)
    return np.vstack(parts)                          # (N, 2**7) statevectors

def fidelity_kernel(A, B=None):
    symmetric = B is None
    K = np.abs(A.conj() @ (A if symmetric else B).T) ** 2   # |<a|b>|^2
    if symmetric:
        K = (K + K.T) / 2.0
        np.fill_diagonal(K, 1.0)
    return K

fm  = ZZFeatureMap(feature_dimension=7, reps=3, entanglement="full")
S   = compute_states(fm, X_train * gamma)            # one simulation per sample
K   = fidelity_kernel(S)                             # N x N quantum kernel
svc = SVC(kernel="precomputed", C=10).fit(K, y_train)
```

**Listing 3.** Core kernel routines (representative excerpt; the repository's `qkernel.py` contains the full version with diagnostics).

# 8. Website / Application

## 8.1 Technologies and Tools Used

The user interface was developed as a modern decoupled full-stack application:

- **Backend API:** Python 3, FastAPI (`api.py`), Uvicorn server with CORS middleware, deployed as a cloud web service on Render.
- **Frontend Application:** Next.js 16.4 (with React 19, TypeScript, and Tailwind CSS), deployed on Vercel.
- **CLI:** `predict.py` for headless and automated batch prediction.

## 8.2 Design and Functionality

The web application implements Algorithm 2 with high responsiveness and accessibility:

1. **Structured Input Groups:** Field inputs are partitioned into Soil Nutrients (N, P, K, pH) and Climate & Environment (Temperature, Humidity, Rainfall).

2. **Synced Native Controls:** Each parameter provides a dual synchronized control: a custom range slider and a numerical entry box with client-side clamping to the model's training boundaries (`FEATURE_RANGES`).
3. **Internationalization:** Full localized UI supporting 5 Indian regional languages (English, Hindi, Tamil, Telugu, and Marathi) with localized crop guidance.
4. **Candidate Probability Breakdown:** Displays the primary recommendation alongside the top 3 alternative candidate crops with decision score bars.
5. **Training Range Coverage:** Real-time visual tracking of whether the user's field parameters lie safely within the quantum model's training distribution or near boundary edges.
6. **Quantum Circuit Metadata:** Displays live backend circuit properties (7 qubits, reps = 3, $C = 10$) and round-trip inference latency ($\approx$ 150–300 ms).

### 8.3 Explanation of the Interface

The desktop layout presents a two-column dashboard: the left panel captures soil/climate inputs, and the right sticky panel dynamically renders recommendation cards, agronomic advice (season, water need, ideal soil), and alternative candidate match scores.

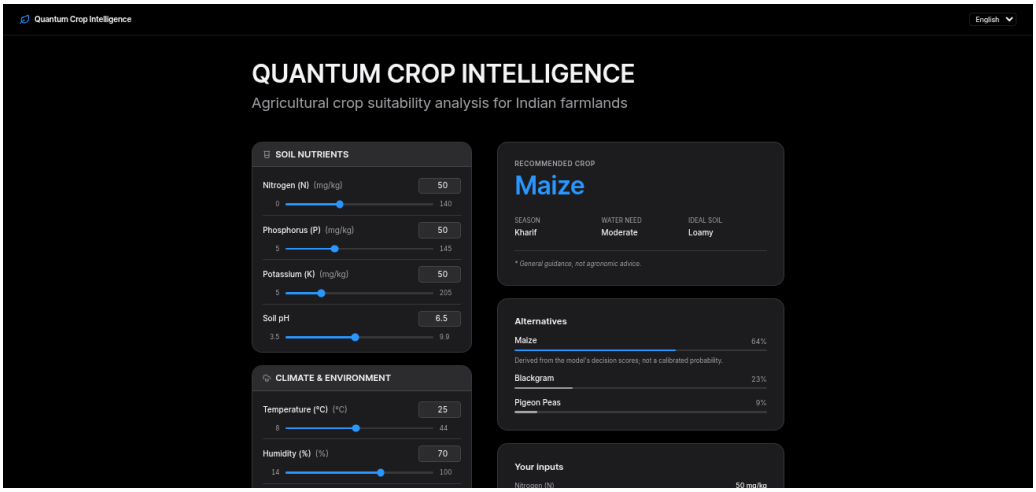



**Figure 2.** Screenshot of the Next.js Quantum Crop Intelligence web interface.

### 8.4 Deployment Architecture

The system operates as a serverless/cloud-native pipeline:

- The Next.js frontend on Vercel communicates asynchronously via client-side fetch with an abort controller (15-second timeout).
- The FastAPI service on Render loads the serialized model pipeline (`qsvc_model.joblib`, scalers, and training statevectors) into memory on startup and computes inference kernel statevectors on demand.

## 9. Results

## 9.1 Experimental Setup

**Table 4.** Final experimental configuration.

| Item | Value |
|---|---|
| Dataset | 2 200 samples, 7 features, 22 balanced classes |
| Split | stratified 80/20, seed 42: 1 760 train / 440 test (20 per class) |
| Preprocessing | `StandardScaler` $\rightarrow$ `MinMaxScaler`$[0, \pi]$, fitted on training data; test clipped |
| Feature map | 7-qubit `ZZFeatureMap`, 3 repetitions, full entanglement |
| Bandwidth $\gamma$ / SVM penalty $C$ | 0.1 / 10 |
| Trainable quantum parameters | 0 |
| Simulation | exact statevector, noiseless, no hardware |
| Search space | 270 configurations (reps $\times$ entanglement $\times$ $\gamma$ $\times$ $C$) |

## 9.2 Input Provided

The program receives the scaled training matrix $X_{\text{train}} \in \mathbb{R}^{1760\times 7}$, the test matrix $X_{\text{test}} \in \mathbb{R}^{440\times 7}$, the integer-encoded labels of the 22 crops, and the hyper-parameter grid of Section 5.2.

## 9.3 Output Obtained

The pipeline outputs the predicted crop for every test sample, the kernel matrices $K_{\text{train}} \in \mathbb{R}^{1760\times 1760}$ and $K_{\text{test}} \in \mathbb{R}^{440\times 1760}$, the selected configuration, accuracy and F1 metrics, a per-class report, confusion matrices and kernel diagnostics.

## 9.4 Model Selection

The configuration *reps* = 3, *full* entanglement, $\gamma = 0.10$, $C = 10$ obtained the highest mean cross-validation accuracy of the 270-point search (Table 5). Several neighbouring configurations lie within one standard deviation, i.e. the ranking among the leading configurations is not sharp.

**Table 5.** Six best configurations of the 270-point search (repeated stratified CV on the training set).

| Reps | Entanglement | $\gamma$ | $C$ | CV mean | CV std |
|---|---|---|---|---|---|
| 3 | full | 0.10 | 10 | 0.98693 | 0.00227 |
| 3 | full | 0.10 | 1000 | 0.98523 | 0.00489 |
| 2 | full | 0.15 | 10 | 0.98466 | 0.00385 |
| 2 | full | 0.05 | 1000 | 0.98466 | 0.00385 |
| 3 | full | 0.02 | 100 | 0.98409 | 0.00385 |
| 3 | full | 0.05 | 100 | 0.98409 | 0.00462 |

## 9.5 Final Performance

**Table 6.** QSVC performance of the selected configuration.

| **Metric** | **Value** |
|---|---|
| Test accuracy (440 samples) | **99.09%** (4 errors) |
| Macro-F1 / weighted-F1 | 0.9909 / 0.9909 |
| Nested-CV accuracy | $98.20\% \pm 0.82\%$ |
| Best full-training CV accuracy | $98.69\% \pm 0.23\%$ |
| Training accuracy | 99.55% |
| 95% Wilson interval for test accuracy[†] | 97.69% – 99.65% |

[†]Computed from 436 correct predictions out of 440.

Training and test accuracy differ by only 0.45 percentage points, and the nested-CV estimate agrees with the test result within one standard deviation, so there is no sign of over-fitting. The selected kernel is well conditioned for learning: off-diagonal mean 0.0882, off-diagonal standard deviation 0.1294, minimum eigenvalue $1.39 \times 10^{-5}$ (positive, hence a valid kernel) and an effective rank of 39.9.

**Per-class results.** Seventeen of the 22 classes obtain F1 = 1.00 (Table 7). The lowest values are jute and rice (0.95) and lentil and maize (0.97). The repository's feature-comparison plots examine the pairs jute/rice, lentil/mothbeans and maize/cotton, which are the pairs whose members carry the imperfect scores; a numeric listing of the four individual misclassifications was not saved.

**Table 7.** Per-class precision (P), recall (R) and F1 on the test set (20 samples per class).

| **Class** | P | R | F1 | **Class** | P | R | F1 |
|---|---|---|---|---|---|---|---|
| apple | 1.00 | 1.00 | 1.00 | mango | 1.00 | 1.00 | 1.00 |
| banana | 1.00 | 1.00 | 1.00 | mothbeans | 0.95 | 1.00 | 0.98 |
| blackgram | 1.00 | 1.00 | 1.00 | mungbean | 1.00 | 1.00 | 1.00 |
| chickpea | 1.00 | 1.00 | 1.00 | muskmelon | 1.00 | 1.00 | 1.00 |
| coconut | 1.00 | 1.00 | 1.00 | orange | 1.00 | 1.00 | 1.00 |
| coffee | 1.00 | 1.00 | 1.00 | papaya | 1.00 | 1.00 | 1.00 |
| cotton | 0.95 | 1.00 | 0.98 | pigeonpeas | 1.00 | 1.00 | 1.00 |
| grapes | 1.00 | 1.00 | 1.00 | pomegranate | 1.00 | 1.00 | 1.00 |
| jute | 0.95 | 0.95 | 0.95 | rice | 0.95 | 0.95 | 0.95 |
| kidneybeans | 1.00 | 1.00 | 1.00 | watermelon | 1.00 | 1.00 | 1.00 |
| lentil | 1.00 | 0.95 | 0.97 | | | | |
| maize | 1.00 | 0.95 | 0.97 | | | | |

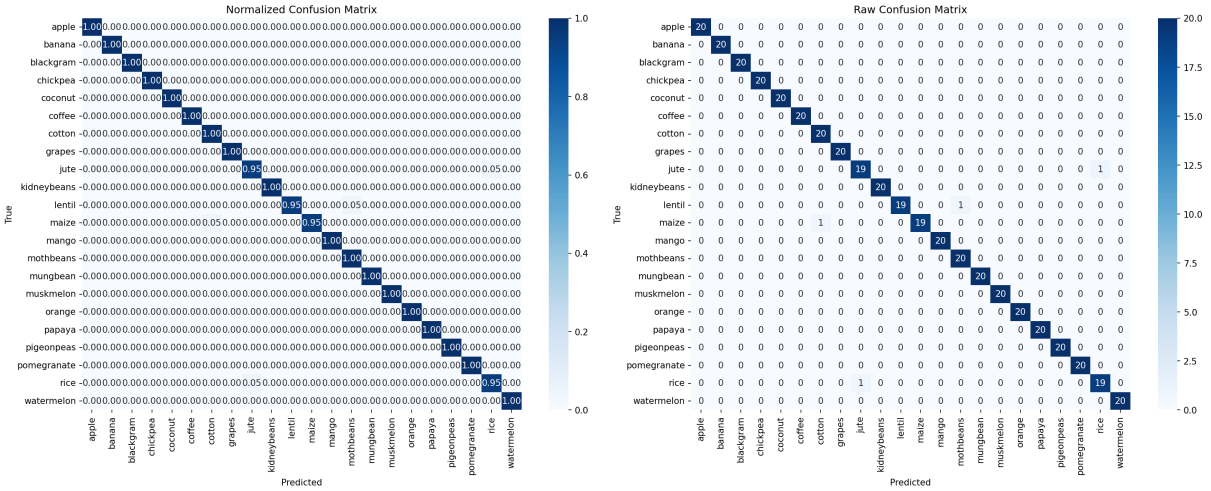



**Figure 3.** Normalised (left) and raw (right) 22-class confusion matrices of the QSVC on the 440 test samples.

## 9.6 Comparison with Classical Models

**Table 8.** QSVC versus tuned classical models on the same split.

| Model | CV accuracy (mean ± std) | Test accuracy | Test errors |
|---|---|---|---|
| Random Forest | $0.9947 \pm 0.0041$ | **0.9955** | 2 |
| **QSVC (this work)** | $0.9820 \pm 0.0082^{*}$ | **0.9909** | 4 |
| Linear SVM | $0.9858 \pm 0.0051$ | 0.9909 | 4 |
| RBF-SVM | $0.9867 \pm 0.0031$ | 0.9886 | 5 |
| $k$-NN | $0.9765 \pm 0.0090$ | 0.9841 | 7 |

*Nested CV, a stricter estimator; the classical CV figures are the best scores of their tuning searches. The like-for-like QSVC figure is $0.9869 \pm 0.0023$.

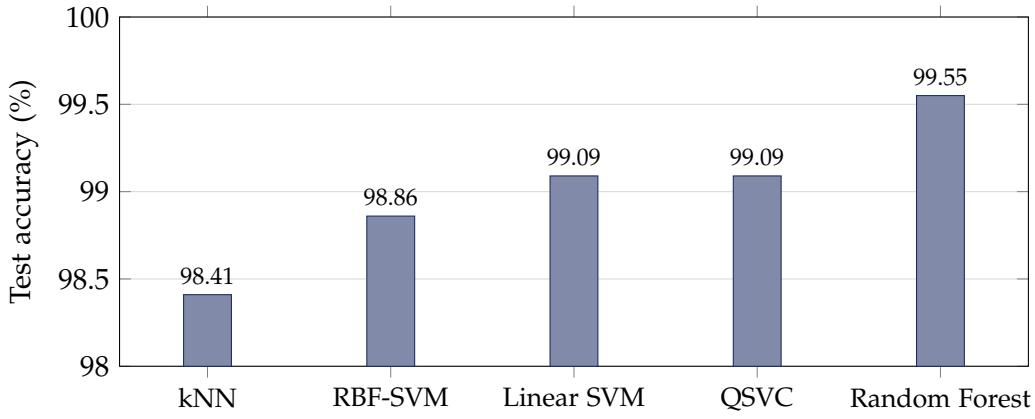



**Figure 4.** Test accuracy of the QSVC and the classical baselines (440 test samples; note the truncated axis).

McNemar's test compares the QSVC with the best classical model, the Random Forest, on the same 440 test samples. The contingency table is $\left(\begin{smallmatrix} 435 & 1 \\ 3 & 1 \end{smallmatrix}\right)$ (435 samples both models classify correctly, 1 and 3 samples on which exactly one of them errs, and 1 sample both misclassify),

giving $\chi^2 = (|1-3|-1)^2/(1+3) = 0.25$ and $p = 0.617$. The difference of two samples is not statistically significant. A plain linear SVM matches the QSVC exactly (99.09%), which indicates that the dataset is close to linearly separable and offers little room for a non-linear quantum feature map to help.

## 9.7 Development History and the Role of the Bandwidth

Table 9 summarises how the design evolved. The numbers for the first prototype and for the corrected development version were recorded from the authors' own runs of `train.py` and `trainv1.py` on an earlier, smaller data split (550 training and 440 test samples).

**Table 9.** Development history.

| Stage | Description | Train | Test | Time |
|---|---|---|---|---|
| VQC | 5 qubits, 50 trainable parameters, COBYLA, 1 500 evaluations (final loss 3.35) | 36.8% | 32.1% | – |
| Prototype (`train.py`) | 7 qubits, reps = 2, full; pairwise `FidelityQuantumKernel`, $C = 1$, no tuning; 550/440 samples | 100% | 65.0% | 2 766.8 s (train) + 2 260.9 s (test) $\approx$ 83.8 min |
| Corrected (`trainv1.py`) | statevector kernel, $\gamma$ and $C$ search over 8 kernels; same 550/440 samples | – | 97.5% (CV 97.3%) | 9.8 s (whole search) |
| Final (`trainv2.py`) | reps = 3, full, $\gamma = 0.1$, $C = 10$; 270 configurations; 1 760/440 samples | 99.5% | 99.1% | not recorded (see text) |

**Why the variational classifier failed.** Training a parameterised circuit with a gradient-free optimiser over 50 parameters did not converge to a useful classifier, in line with the known difficulty of optimising variational circuits [5]. The kernel approach removes this training loop.

**Why the prototype over-fitted.** With the untuned prototype the SVM reproduced the training set perfectly yet reached 65.0% on the test set (macro-F1 0.67; weakest classes pigeonpeas 0.36, mothbeans 0.41, papaya 0.42). A perfect training score combined with poor generalisation is the signature of a kernel whose off-diagonal entries are too small for samples to share information, i.e. kernel concentration. The corrected development runs support this explanation: increasing the bandwidth lowers the off-diagonal kernel mean and the accuracy together (Table 10, Figure 5). For reps $= 2$, accuracy falls from 97.1% at $\gamma = 0.10$ to 60.5% at $\gamma = 1.0$ while the off-diagonal mean falls from 0.116 to 0.010. The prototype itself did not log its kernel statistics, so the link between those statistics and its 65% result is an inference, not a measurement.

**Table 10.** Development sweep on the 550/440 split (`trainv1.py`): best 5-fold CV accuracy over $C \in \{1, 10, 100, 1000\}$ and mean off-diagonal kernel value.

| | reps = 1 | | reps = 2 | |
|---|---|---|---|---|
| $\gamma$ | CV acc. | off-diag. mean | CV acc. | off-diag. mean |
| 0.10 | 97.3% | 0.2401 | 97.1% | 0.1162 |
| 0.25 | 97.3% | 0.0618 | 95.5% | 0.0210 |
| 0.50 | 95.3% | 0.0253 | 86.9% | 0.0111 |
| 1.00 | 91.5% | 0.0172 | 60.5% | 0.0098 |

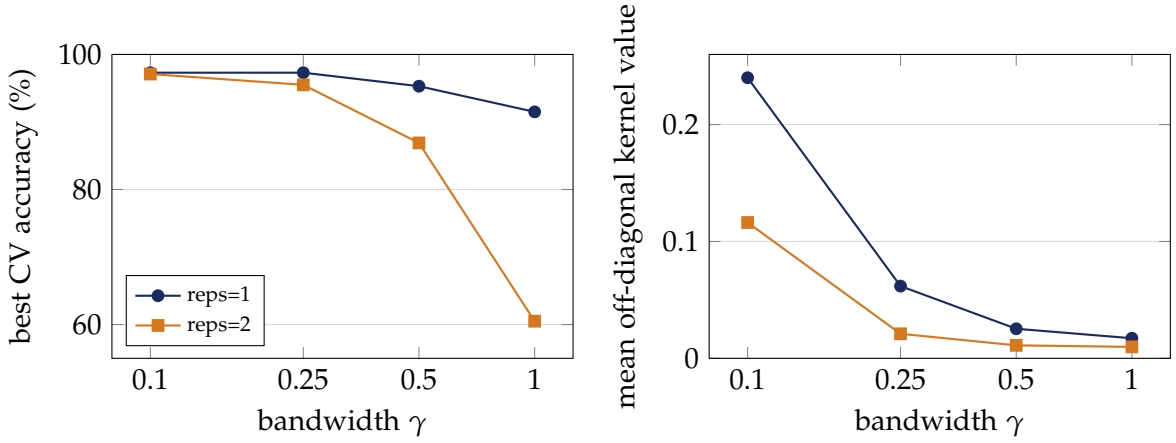



**Figure 5.** Effect of the bandwidth $\gamma$ in the development runs (550/440 split). Large $\gamma$ drives the off-diagonal kernel values towards zero and accuracy collapses.

## 9.8 Algorithm Performance

Summarising the evidence: the QSVC classifies 99.09% of unseen samples correctly, generalises (train 99.55%, nested CV 98.20%, test 99.09%), and is statistically indistinguishable from the best tuned classical model. The two engineering changes that mattered most were the statevector formulation of the kernel (speed) and the bandwidth plus tuning (generalisation). The pipeline's quantum part is exact simulation, so these results characterise the algorithm, not the behaviour of noisy hardware.

## 9.9 Time Complexity

Let $n$ be the number of qubits (7), $r$ the number of feature-map repetitions (3), $N$ the number of training samples (1 760), $M$ the number of test samples (440) and $G$ the number of hyper-parameter configurations.

**Circuit size.** Decomposed into the gates $\{H, P, \mathrm{CX}\}$, one repetition of the full-entanglement feature map contains $n$ Hadamards, $n$ single-qubit phases and, for each of the $\binom{n}{2}$ pairs, two CX gates and one phase, i.e. $O(n^2)$ gates. Table 11 gives the exact counts for $n = 7$, obtained by transpiling the Qiskit 1.1.0 circuit without optimisation.

**Table 11.** Exact gate counts and depth of the 7-qubit full-entanglement ZZ feature map (Qiskit 1.1.0, basis $\{H, P, \mathrm{CX}\}$, no optimisation).

| Reps | H | P | CX | Total gates | Depth |
|---|---|---|---|---|---|
| 1 | 7 | 28 | 42 | 77 | 35 |
| 2 | 14 | 56 | 84 | 154 | 58 |
| **3 (selected)** | 21 | 84 | 126 | **231** | **81** |

**Stage costs.** Applying a gate to a $2^n$-amplitude statevector costs $O(2^n)$, so simulating one sample costs $O(r\, n^2\, 2^n)$ and all $N$ samples cost $O(N\, r\, n^2\, 2^n)$. The kernel matrix is the product of an $N \times 2^n$ and a $2^n \times N$ matrix, $O(N^2\, 2^n)$. SVM training with a precomputed kernel (SMO, one-vs-one) costs between $O(N^2)$ and $O(N^3)$ in practice [4]. Predicting $M$ new samples costs

$O(M\,r\,n^2\,2^n)$ for the states plus $O(M\,N\,2^n)$ for the kernel rows. Model selection multiplies the cost by the number of cached kernels (54) and by the number of SVM fits in the nested cross-validation ($G = 270$ configurations).

**Table 12.** Asymptotic time and space complexity of the QSVC pipeline.

| Stage | Time | Space |
|---|---|---|
| Encode one sample (statevector) | $O(r\,n^2\,2^n)$ | $O(2^n)$ |
| Encode all $N$ training samples | $O(N\,r\,n^2\,2^n)$ | $O(N\,2^n)$ |
| Training kernel $K$ ($N \times N$) | $O(N^2\,2^n)$ | $O(N^2)$ |
| SVM training (one-vs-one, SMO) | $O(N^2)$ to $O(N^3)$ | $O(N^2)$ |
| Predict $M$ new samples | $O(M\,r\,n^2\,2^n + M\,N\,2^n)$ | $O(M\,N)$ |

**Gain over pairwise evaluation.** The prototype evaluates one circuit per *pair* of samples, each comprising the feature map and its inverse, at a cost of $O(N^2\,r\,n^2\,2^n)$; in the 550/440 run this means $550^2 + 440 \cdot 550 = 544\,500$ circuits of roughly $2 \times 154 = 308$ gates. The statevector formulation needs only $550 + 440 = 990$ simulations and one matrix product. The factor $r\,n^2$ per kernel entry disappears, and the measured effect was a reduction of the kernel time from 5 027.7 s (83.8 min) to well under a minute: in the corrected development runs a complete 550-sample kernel took between 0.5 s and 3.4 s (the first call included start-up cost) and the entire eight-configuration search took 9.8 s. A structured wall-clock benchmark of the final `trainv2.py` run on the full 1 760/440 data was not recorded; it is listed as future work.

### 9.10 Space Complexity

A statevector holds $2^7 = 128$ complex numbers (16 bytes each), i.e. 2 048 bytes per sample. For $N = 1\,760$ the training states occupy $\approx 3.6$ MB, the training kernel ($1760^2$ doubles) $\approx 24.8$ MB and the test kernel ($440 \times 1760$ doubles) $\approx 6.2$ MB, so the whole computation fits comfortably in memory. Asymptotically the space is $O(N\,2^n + N^2)$; the $2^n$ factor is a property of classical simulation, and memory would grow exponentially with the number of qubits.

### 9.11 Observations

- The statevector kernel and a tuned bandwidth turned a 65%, 84-minute prototype into a 99%, seconds-scale pipeline.
- The best bandwidth lay at a small value ($\gamma = 0.1$) and the optimum of the $\gamma$ grid was not at its upper edge; because $\gamma = 0.02$ was the lower edge of the grid, values below it were not explored.
- At small $\gamma$ the encoding angles are small, and the QSVC behaves like a smooth similarity-based classifier whose accuracy matches that of a linear SVM.
- Classical models are near-perfect on this dataset, so the data cannot reveal a quantum benefit even if one exists.

## 10. Conclusion

**How the problem was addressed.** We built a hybrid quantum-classical classifier that recommends one of 22 crops from seven soil and climate measurements. A fixed 7-qubit ZZ feature

map encodes each sample, the fidelity of the encoded states defines a quantum kernel, and a classical SVM learns from the kernel matrix. The initial approaches failed for identifiable reasons: a variational circuit did not train (32.1% test accuracy), and an untuned pairwise kernel over-fitted (100% train, 65.0% test) and was slow. Computing each sample's statevector once, adding a bandwidth parameter, and selecting hyper-parameters by nested cross-validation fixed both problems.

**Was the solution successful?** Yes, for the stated problem. The final model reaches **99.09%** accuracy and macro-F1 0.9909 on 440 unseen samples (nested-CV estimate $98.20\% \pm 0.82\%$), with 17 of 22 crops classified perfectly, and the training pipeline now runs in seconds on a standard computer.

**Key results.** (i) Bandwidth and tuning are decisive for quantum kernels: large $\gamma$ concentrates the kernel and destroys accuracy. (ii) The QSVC matches the best classical models: Random Forest 99.55%, linear SVM 99.09%, RBF-SVM 98.86%, $k$-NN 98.41%; McNemar's test gives $p = 0.617$. (iii) The exact feature map of the selected model has 231 gates and depth 81.

**How effectively the algorithm performed.** The algorithm is effective and reliable on this dataset, reaching classical-level accuracy with zero trainable quantum parameters. It is *not* shown to outperform classical methods: the dataset is easy (even a linear SVM is near-perfect), and all quantum results are noiseless simulations. We therefore claim a working, well-evaluated quantum-kernel pipeline and make no claim of quantum advantage, speed advantage or hardware validation.

**Limitations and future work.** The main results use a single split of a static benchmark dataset, 20 test samples per class and exact simulation. Future work includes finite-shot and noise-model studies followed by a hardware run, harder datasets on which classical kernels struggle, trainable kernels (kernel-target alignment), a structured runtime benchmark of the final pipeline, and packaging the trained artefacts so that the application can be run from a fresh clone.

## References

[1] V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.

[2] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Phys. Rev. Lett.*, vol. 122, 040504, 2019.

[3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[4] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, art. 27, 2011.

[5] J. R. McClean et al., "Barren plateaus in quantum neural network training landscapes," *Nat. Commun.*, vol. 9, 4812, 2018.

[6] S. Thanasilp et al., "Exponential concentration in quantum kernel methods," *Nat. Commun.*, vol. 15, 5200, 2024.

[7] R. Shaydulin and S. M. Wild, "Importance of kernel bandwidth in quantum machine learning," *Phys. Rev. A*, vol. 106, 042407, 2022.

[8] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.

[9] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, art. 91, 2006.

[10] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, pp. 153–157, 1947.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] Qiskit Contributors, "Qiskit: An open-source framework for quantum computing," 2023. https://qiskit.org.

[13] A. Ingle, "Crop Recommendation Dataset," Kaggle (file obtained via the public Harvestify repository).